

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-fullscreen">
<title>Rak'ah Counter</title>
<style>
  * { margin: 0; padding: 0; box-sizing: border-box; -webkit-tap-highlight-color: transparent; }

  :root {
    --bg: #000;
    --fg: #fff;
    --dim: rgba(255,255,255,0.18);
    --accent: rgba(255,255,255,0.08);
  }

  html, body {
    width: 100%; height: 100%;
    background: var(--bg);
    color: var(--fg);
    font-family: 'Georgia', serif;
    overflow: hidden;
    touch-action: none;
  }

  /* —— SCREENS —— */
  .screen {
    position: fixed; inset: 0;
    display: flex; flex-direction: column;
    align-items: center; justify-content: center;
    transition: opacity 0.4s ease;
  }
  .screen.hidden { opacity: 0; pointer-events: none; }

  /* —— SETUP SCREEN —— */
  #setup {
```

```
gap: 0;
padding: 48px 32px 60px;
justify-content: space-between;
}
```

```
.app-title {
font-size: clamp(22px, 6vw, 30px);
font-weight: 300;
letter-spacing: 0.18em;
text-transform: uppercase;
color: var(--fg);
text-align: center;
margin-top: env(safe-area-inset-top, 20px);
}
```

```
.app-subtitle {
font-size: 13px;
letter-spacing: 0.12em;
color: var(--dim);
text-align: center;
margin-top: 6px;
}
```

```
.select-label {
font-size: 11px;
letter-spacing: 0.2em;
text-transform: uppercase;
color: var(--dim);
text-align: center;
margin-bottom: 20px;
}
```

```
.rakah-options {
display: flex;
gap: 16px;
justify-content: center;
width: 100%;
}
```

```
.rakah-btn {
  flex: 1;
  max-width: 90px;
  aspect-ratio: 1;
  border-radius: 50%;
  border: 1px solid rgba(255,255,255,0.25);
  background: transparent;
  color: var(--fg);
  font-size: 32px;
  font-family: 'Georgia', serif;
  font-weight: 300;
  cursor: pointer;
  transition: all 0.2s ease;
  display: flex; align-items: center; justify-content: center;
}
.rakah-btn.selected {
  border-color: #fff;
  background: rgba(255,255,255,0.12);
  box-shadow: 0 0 28px rgba(255,255,255,0.12);
}
.rakah-btn:active { transform: scale(0.93); }
```

```
.start-btn {
  width: 100%;
  max-width: 280px;
  padding: 18px 0;
  border-radius: 60px;
  border: 1px solid rgba(255,255,255,0.5);
  background: transparent;
  color: var(--fg);
  font-size: 15px;
  font-family: 'Georgia', serif;
  letter-spacing: 0.22em;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 0.25s ease;
  margin-bottom: env(safe-area-inset-bottom, 20px);
}
```

```
.start-btn:not(:disabled):active {
  background: rgba(255,255,255,0.1);
  transform: scale(0.97);
}
.start-btn:disabled {
  opacity: 0.25;
  cursor: default;
}

/* ——— COUNTER SCREEN ——— */
#counter {
  background: #000;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

#rakah-number {
  font-size: clamp(200px, 55vw, 320px);
  font-family: 'Georgia', serif;
  font-weight: 400;
  color: #fff;
  line-height: 1;
  letter-spacing: -0.02em;
  user-select: none;
  transition: opacity 0.15s ease;
}

.counter-label {
  font-size: 11px;
  letter-spacing: 0.22em;
  text-transform: uppercase;
  color: rgba(255,255,255,0.2);
  margin-top: 16px;
  user-select: none;
}

/* Pose indicator — subtle dots at bottom */
```

```
.pose-bar {
  position: fixed;
  bottom: calc(env(safe-area-inset-bottom, 16px) + 24px);
  left: 50%;
  transform: translateX(-50%);
  display: flex;
  gap: 8px;
  align-items: center;
}
.pose-dot {
  width: 6px; height: 6px;
  border-radius: 50%;
  background: rgba(255,255,255,0.15);
  transition: background 0.3s ease;
}
.pose-dot.active { background: rgba(255,255,255,0.7); }
.pose-dot.done { background: rgba(255,255,255,0.35); }
```

/* Stop button — tiny, top right */

```
.stop-btn {
  position: fixed;
  top: calc(env(safe-area-inset-top, 16px) + 12px);
  right: 20px;
  background: transparent;
  border: 1px solid rgba(255,255,255,0.2);
  color: rgba(255,255,255,0.3);
  font-size: 10px;
  letter-spacing: 0.18em;
  text-transform: uppercase;
  padding: 6px 12px;
  border-radius: 20px;
  cursor: pointer;
  font-family: 'Georgia', serif;
  transition: all 0.2s ease;
}
.stop-btn:active { color: #fff; border-color: #fff; }
```

/* hidden video element for camera feed */

```
#camera-feed {
  position: fixed;
  width: 1px; height: 1px;
  opacity: 0;
  pointer-events: none;
  top: -999px; left: -999px;
}

/* Loading state */
#loading-overlay {
  position: fixed; inset: 0;
  background: #000;
  display: flex; flex-direction: column;
  align-items: center; justify-content: center;
  gap: 20px;
  z-index: 100;
  opacity: 0; pointer-events: none;
  transition: opacity 0.3s;
}
#loading-overlay.visible { opacity: 1; pointer-events: all; }
.loading-text {
  font-size: 12px;
  letter-spacing: 0.2em;
  text-transform: uppercase;
  color: rgba(255,255,255,0.4);
}
.loading-spinner {
  width: 28px; height: 28px;
  border: 1px solid rgba(255,255,255,0.15);
  border-top-color: rgba(255,255,255,0.6);
  border-radius: 50%;
  animation: spin 1s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
</style>
</head>
<body>
```

```

<!-- Hidden camera video -->
<video id="camera-feed" autoplay playsinline muted></video>

<!-- Loading overlay -->
<div id="loading-overlay">
  <div class="loading-spinner"></div>
  <div class="loading-text">Preparing camera...</div>
</div>

<!-- SETUP SCREEN -->
<div class="screen" id="setup">
  <div>
    <div class="app-title">Rak'ah Counter</div>
    <div class="app-subtitle">Camera · On-device · Private</div>
  </div>

  <div>
    <div class="select-label">Select rak'ahs</div>
    <div class="rakah-options">
      <button class="rakah-btn" data-count="2">2</button>
      <button class="rakah-btn" data-count="3">3</button>
      <button class="rakah-btn" data-count="4">4</button>
    </div>
  </div>

  <button class="start-btn" id="start-btn" disabled>Begin</button>
</div>

<!-- COUNTER SCREEN -->
<div class="screen hidden" id="counter">
  <div id="rakah-number">1</div>
  <div class="counter-label">Rak'ah</div>

  <!-- 7 pose steps: Stand · Ruku · Stand · Sajdah · Sit · Sajdah · Stand -->
  <div class="pose-bar" id="pose-bar">
    <div class="pose-dot" data-step="0"></div>
    <div class="pose-dot" data-step="1"></div>
    <div class="pose-dot" data-step="2"></div>

```

```
<div class="pose-dot" data-step="3"></div>
<div class="pose-dot" data-step="4"></div>
<div class="pose-dot" data-step="5"></div>
<div class="pose-dot" data-step="6"></div>
</div>
```

```
<button class="stop-btn" id="stop-btn">Stop</button>
```

```
<!-- DEBUG OVERLAY -->
```

```
<div id="debug-overlay" style="position:fixed;top:60px;left:10px;right:10px;font-
family:monospace;font-size:11px;color:#0f0;background:rgba(0,0,0,0.6);padding:8px;border-
radius:6px;line-height:1.5;white-space:pre;z-index:999;"></div>
</div>
```

```
<!-- TensorFlow.js + MoveNet -->
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/tensorflow/4.17.0/tf.min.js"></script>
```

```
<script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/pose-detection@2.1.3/dist/pose-
detection.min.js"></script>
```

```
<script>
```

```
//
```

```
// STATE
```

```
//
```

```
let selectedCount = 0;
let currentRakah = 1;
let poseStep = 0; // 0–6 within one rak'ah cycle
let totalTarget = 0;
let detector = null;
let rafId = null;
let isRunning = false;
let modelReady = false;
```

```
// Pose sequence per rak'ah:
```

```
// 0=Standing 1=Ruku 2=Standing 3=Sajdah 4=Sitting 5=Sajdah 6=Standing(next)
```



```
const POSE_NAMES = ['Standing','Ruku','Standing','Sajdah','Sitting','Sajdah','Standing'];
```

```
// Minimum frames a pose must be held before it counts
```

```
const HOLD_FRAMES = 8;
```

```
let holdCount = 0;
```

```
let lastPose = null;
```

```
//
```

```
// UI ELEMENTS
```

```
//
```

```
const setupScreen = document.getElementById('setup');
```

```
const counterScreen = document.getElementById('counter');
```

```
const rakahNumber = document.getElementById('rakah-number');
```

```
const doneOverlay = document.getElementById('done-overlay');
```

```
const doneNumber = document.getElementById('done-number');
```

```
const startBtn = document.getElementById('start-btn');
```

```
const stopBtn = document.getElementById('stop-btn');
```

```
const loadingOverlay = document.getElementById('loading-overlay');
```

```
const poseDots = document.querySelectorAll('.pose-dot');
```

```
const videoEl = document.getElementById('camera-feed');
```

```
//
```

```
// SETUP SCREEN LOGIC
```

```
//
```

```
document.querySelectorAll('.rakah-btn').forEach(btn => {
```

```
  btn.addEventListener('click', () => {
```

```
    document.querySelectorAll('.rakah-btn').forEach(b => b.classList.remove('selected'));
```

```
    btn.classList.add('selected');
```

```
    selectedCount = parseInt(btn.dataset.count);
```

```
    startBtn.disabled = false;
```

```
});  
});
```

```
startBtn.addEventListener('click', async () => {  
  if (!selectedCount) return;  
  totalTarget = selectedCount;  
  currentRakah = 1;  
  poseStep = 0;  
  holdCount = 0;  
  lastPose = null;  
  await startSession();  
});
```

```
stopBtn.addEventListener('click', stopSession);
```

```
//
```

```
// SESSION START / STOP
```

```
//
```

```
async function startSession() {  
  showLoading(true);  
  
  try {  
    // Start camera (front-facing, no audio)  
    const stream = await navigator.mediaDevices.getUserMedia({  
      video: {  
        facingMode: 'user',  
        width: { ideal: 320 },  
        height: { ideal: 480 }  
      },  
      audio: false  
    });  
    videoEl.srcObject = stream;  
    await new Promise(res => { videoEl.onloadedmetadata = res; });  
    await videoEl.play();
```

```

// Load MoveNet if not yet loaded
if (!detector) {
  detector = await poseDetection.createDetector(
    poseDetection.SupportedModels.MoveNet,
    { modelType: poseDetection.movenet.modelType.SINGLEPOSE_LIGHTNING }
  );
}

modelReady = true;
showLoading(false);

// Switch screens
setupScreen.classList.add('hidden');
counterScreen.classList.remove('hidden');

updateDisplay();
updatePoseDots();
isRunning = true;
detectLoop();

} catch (err) {
  showLoading(false);
  alert('Camera access is required. Please allow camera permission and try again.');
```

console.error(err);

```

}
}

function stopSession() {
  isRunning = false;
  if (rafId) { cancelAnimationFrame(rafId); rafId = null; }

  // Stop camera tracks
  if (videoEl.srcObject) {
    videoEl.srcObject.getTracks().forEach(t => t.stop());
    videoEl.srcObject = null;
  }
}

```

```
counterScreen.classList.add('hidden');
setupScreen.classList.remove('hidden');
}
```

```
//
```

```
// DETECTION LOOP
//
```

```
async function detectLoop() {
  if (!isRunning) return;

  try {
    if (videoEl.readyState >= 2 && detector) {
      const poses = await detector.estimatePoses(videoEl);
      if (poses.length > 0) {
        const pose = classifyPose(poses[0].keypoints);
        processPose(pose);
      }
    }
  } catch(e) { /* swallow */ }

  rafId = requestAnimationFrame(detectLoop);
}
```

```
//
```

```
// POSE CLASSIFICATION
// Uses keypoint y-positions to distinguish postures
// Camera is on floor/rug pointing up at person
//
```

```
function getKp(keypoints, name) {
  return keypoints.find(k => k.name === name);
}
```

```
}
```

```
function classifyPose(keypoints) {  
  // Key joints  
  const nose    = getKp(keypoints, 'nose');  
  const lShoulder = getKp(keypoints, 'left_shoulder');  
  const rShoulder = getKp(keypoints, 'right_shoulder');  
  const lHip     = getKp(keypoints, 'left_hip');  
  const rHip     = getKp(keypoints, 'right_hip');  
  const lKnee    = getKp(keypoints, 'left_knee');  
  const rKnee    = getKp(keypoints, 'right_knee');  
  const lAnkle   = getKp(keypoints, 'left_ankle');  
  const rAnkle   = getKp(keypoints, 'right_ankle');  
  
  // Need minimum confidence  
  const minConf = 0.25;  
  const valid = [nose, lShoulder, rShoulder, lHip, rHip].every(  
    k => k && k.score >= minConf  
  );  
  if (!valid) return null;  
  
  const shoulderY = avg(lShoulder.y, rShoulder.y);  
  const hipY      = avg(lHip.y, rHip.y);  
  const kneeY     = lKnee && rKnee && lKnee.score > minConf && rKnee.score > minConf  
    ? avg(lKnee.y, rKnee.y) : null;  
  const ankleY    = lAnkle && rAnkle && lAnkle.score > minConf && rAnkle.score > minConf  
    ? avg(lAnkle.y, rAnkle.y) : null;  
  const noseY     = nose.y;  
  
  // Camera is on floor pointing up, so "down" in image = higher on body  
  // y=0 is top of image frame (head when standing seen from below)  
  // When person stands: nose near top (low y), hips mid, ankles at bottom (high y)  
  // When person does ruku (bowing): torso horizontal → shoulder y ≈ hip y  
  // When person does sajdah (prostration): nose/forehead touches ground → very low in frame  
  // When sitting: knees bent, hips low but not as low as sajdah  
  
  const frameH = videoEl.videoHeight || 480;  
  const noseRel = noseY / frameH;    // 0=top, 1=bottom
```

```

const shoulderRel = shoulderY / frameH;
const hipRel = hipY / frameH;
const kneeRel = kneeY ? kneeY / frameH : null;

// Torso lean: difference between shoulder and hip y
const torsoVertical = Math.abs(shoulderRel - hipRel);

let detected = 'Standing';
if (noseRel < 0.28 && torsoVertical < 0.18) {
  detected = 'Sajdah';
} else if (torsoVertical < 0.14 && noseRel > 0.25) {
  detected = 'Ruku';
} else if (kneeRel !== null && kneeRel < hipRel - 0.08 && noseRel > 0.35) {
  detected = 'Sitting';
}

// DEBUG OUTPUT
const dbg = document.getElementById('debug-overlay');
if (dbg) {
  dbg.textContent =
    `nose: ${noseRel.toFixed(2)} shoulder: ${shoulderRel.toFixed(2)} hip: ${hipRel.toFixed(2)}\n` +
    `knee: ${kneeRel !== null ? kneeRel.toFixed(2) : '--'} torsoVert: ${torsoVertical.toFixed(2)}\n` +
    `DETECTED: ${detected} | step: ${poseStep} expects: ${SEQUENCE[poseStep]}\n` +
    `hold: ${holdCount}/${HOLD_FRAMES}`;
}

return detected;
}

function avg(a, b) { return (a + b) / 2; }

//
=====

// POSE STATE MACHINE
//
// Every rak'ah follows the same 6-step movement cycle:
// Step 0: Standing (start / already here at beginning of rak'ah)

```

```

// Step 1: Ruku    (bowing)
// Step 2: Standing (rise from ruku)
// Step 3: Sajdah  (first prostration)
// Step 4: Sitting (sit between the two sajdahs)
// Step 5: Sajdah  (second prostration)
// Step 6: Standing ← this standing completes the rak'ah
//                AND immediately begins the next rak'ah (counter +1)
//
// EXCEPTION — after the final rak'ah there is NO standing:
// the person sits for Tashahhud/Salam instead.
// The app detects this final Sitting after Sajdah 5 on the last rak'ah
// and simply holds the final number. Camera keeps running; nothing changes on screen.
//
// TASHAHHUD SIT (mid-prayer, 2-rak'ah break in 3 or 4 rak'ah prayers):
// After rak'ah 2 of a 3- or 4-rak'ah prayer the person sits longer before
// standing for rak'ah 3. The state machine handles this naturally because
// step 6 (Standing) is what triggers the next rak'ah — the sitting duration
// doesn't matter; the app just waits for the next Standing.
//

```

```

// The 7-step sequence (index 0–6). Index 6 = "Standing that starts next rak'ah"
const SEQUENCE = ['Standing','Ruku','Standing','Sajdah','Sitting','Sajdah','Standing'];

```

```

function processPose(pose) {
  if (!pose) return;

```

```

  // Require pose to be held for HOLD_FRAMES consecutive frames before acting
  if (pose === lastPose) {
    holdCount++;
  } else {
    holdCount = 0;
    lastPose = pose;
    return;
  }

```

```

  if (holdCount !== HOLD_FRAMES) return; // only fire once per hold, not every frame

```

```
const currentExpected = SEQUENCE[poseStep];
```

```
// Ignore if already in the current expected pose  
if (pose === currentExpected) return;
```

```
// Only advance if this pose matches the NEXT expected step  
const nextStep = poseStep + 1;  
if (nextStep >= SEQUENCE.length) return;
```

```
if (pose === SEQUENCE[nextStep]) {  
  poseStep = nextStep;  
  updatePoseDots();  
}
```

```
// — Step 6: Standing after Sajdah 2 → rak'ah is complete  
if (poseStep === 6) {  
  onRakahComplete();  
}
```

```
// — Step 4 (Sitting) on the FINAL rak'ah: person will not stand again  
// The app just stays here and holds the display. Nothing to do — by design.  
}  
}
```

```
function onRakahComplete() {  
  if (currentRakah < totalTarget) {  
    // More rak'ahs to go: increment counter and reset pose cycle.  
    // The Standing at step 6 doubles as step 0 of the next rak'ah,  
    // so we reset poseStep to 0 — the person is already standing.  
    currentRakah++;  
    poseStep = 0;  
    holdCount = 0;  
    updateDisplay();  
    updatePoseDots();  
  }  
  // If currentRakah === totalTarget: prayer is finished.  
  // Do nothing — just hold the final number on screen.  
  // Camera stays on but no UI change. User ends with Salam naturally.
```



```

}

//
=====

// DISPLAY
//
=====

function updateDisplay() {
  rakahNumber×textContent = currentRakah;
}

function updatePoseDots() {
  poseDots.forEach((dot, i) => {
    dot.classList.remove('active', 'done');
    if (i < poseStep) dot.classList.add('done');
    else if (i === poseStep) dot.classList.add('active');
  });
}

// Prayer ends naturally — final number stays on screen.
// User ends with Salam and taps Stop to return to setup.

function showLoading(show) {
  loadingOverlay.classList.toggle('visible', show);
}

// Keep screen awake (Wake Lock API)
async function requestWakeLock() {
  try {
    if ('wakeLock' in navigator) {
      await navigator.wakeLock.request('screen');
    }
  } catch(e) { /* not critical */ }
}

requestWakeLock();
document.addEventListener('visibilitychange', () => {

```

```
    if (document.visibilityState === 'visible') requestWakeLock();  
  });  
</script>  
</body>  
</html>
```